



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

SC4001 Neural Networks

Group Project: Oxford Flower Classification

Name	Matriculation Number	Contribution
Low Jo Yi, Nicole	U2321370D	33.3%
Lu ShanShan	U2320618J	33.3%
Tan Pei Wen Jamie	U2321290C	33.3%

Contents

1 Introduction	3
1.1 Dataset	3
2 Related Works	3
2.1 Traditional Computer Vision	3
2.2 Deep Learning	3
2.2.1 Multi-Layer Perceptron (MLP)	3
2.2.2 Convolution Neural Network (CNN)	3
3 Baseline Model	3
3.1 Optimal Learning Rate	4
3.2 Result	4
4 Modifications of Previously Published Architectures	5
4.1 Multi-layer Perceptron	5
4.1.1 Results Analysis	5
4.2 Visual Prompt Tuning	6
4.2.1 Implementation	6
4.2.2 Results Analysis	7
4.3 Few-shot Learning	7
4.3.1 5-shot Learning	8
4.3.2 Result Analysis	8
5 Advanced techniques	8
5.1 Mixup data Augmentation technique	8
5.1.1 Implementation	8
5.1.2. Result Analysis	9
5.2 Triplet Loss Technique	9
5.2.1 Implementation	10
5.2.2 Model Architecture	10
5.2.3 Result Analysis	11
6 Discussion	11
6.1 Limitations	12
6.1.1 Computer resources	12
6.1.2 Dataset limitation	12
6.2 Future work	12
7 References	13

1 Introduction

The application of image classification is vast, spanning across various sectors including healthcare and manufacturing, for the purpose of clinical diagnosis and defect identification, respectively. Moreover, with the advancements of deep neural network architectures and more specifically, the emergence of Generative Adversarial Network (GAN) allows the generation of realistic synthetic images. As a result, the problem of image classification is exacerbated, highlighting the need for reliable image classification.

This report will explore related works in the domain of image classification as well in-depth analysis of the various techniques that can be implemented to improve the robustness of a classification model.

1.1 Dataset

The Oxford Flowers 102 dataset is used for the classification task, a collection of 102 flower categories occurring in the United Kingdom. The dataset provides a foundation to explore images with varying scale, pose, and light variations, with large variation within categories and some with high similarities across categories

2 Related Works

This section will explain state-of-the-art approaches used for image classification.

2.1 Traditional Computer Vision

Techniques such as thresholding, edge detection and feature descriptors are used in manual feature extraction to identify meaningful textures, shapes and patterns within an image. They are then fed into traditional machine models such as K-Nearest Neighbour (KNN) and Support Vector Machine (SVM) to perform classification. While this method leads to highly interpretable results (GeeksforGeeks, 2024), it is limited as it is unable to handle raw inputs of images and relies heavily on manual feature engineering, which is hard to guarantee for handcrafted features in every image.

2.2 Deep Learning

2.2.1 Multi-Layer Perceptron (MLP)

This method uses layers of perceptron that can capture complex relationships between pixels of an image for high level classification task. While images may be flattened and input into the neural network, spatial information is lost (Ginesh, 2019), causing all pixels.

2.2.2 Convolution Neural Network (CNN)

CNNs can handle direct inputs of images, preserve spatial information through convolutional kernels that extract meaningful local features, automating the feature extraction process. Fully connected layers after the layers of convolution networks rely on these feature maps to classify the images by identifying complex relationships between these patterns.

3 Baseline Model

This section will explain the architecture and technical details of the baseline model.

Since the task deals with image data, a Convolutional Neural Network (CNN) is most suitable for capturing spatial relationships between pixels of an image. However, training a deep neural network can be very computationally intensive and require large amounts of data. Hence, a transfer learning approach using ResNet-50 is used in the baseline model implementation.

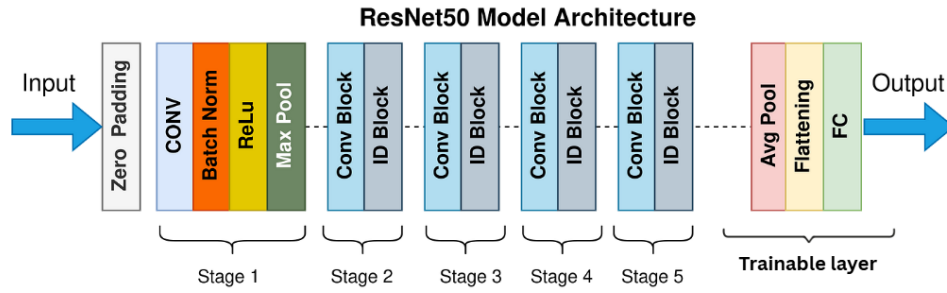


Figure 1: Baseline model architecture

Convolution layers in the original ResNet-50 model were frozen to preserve the kernels (weights) to preserve useful feature representations that are relevant in general image recognition, as identified from images from ImageNet, which were used to train the original ResNet-50 model. Through these kernels, relevant visual features (feature maps) can be extracted when images from the flower dataset are passed through the CNN layers.

Subsequently, these feature maps undergo average pooling and are flattened to pass through a new trainable, fully connected layer, which serves as the final classification head. Only a single (linear) fully connected layer is used to keep the classification head simple, as well as an initial observation that its classification accuracy is good with only a single fully connected layer.

3.1 Optimal Learning Rate

The training was allowed to run for a maximum of 100 epochs with 5-fold cross-validation across varying learning rates. The criteria for selecting the appropriate learning rate were to select the learning rate that provided the highest mean validation accuracy, as it indicates the most stable and effective convergence. Additionally, early stopping was implemented to prevent overfitting, and a regularisation technique, weight decay, was also implemented to promote stable convergence.

From the figure below, the optimal learning rate selected and will be used for the baseline model is 0.0005.

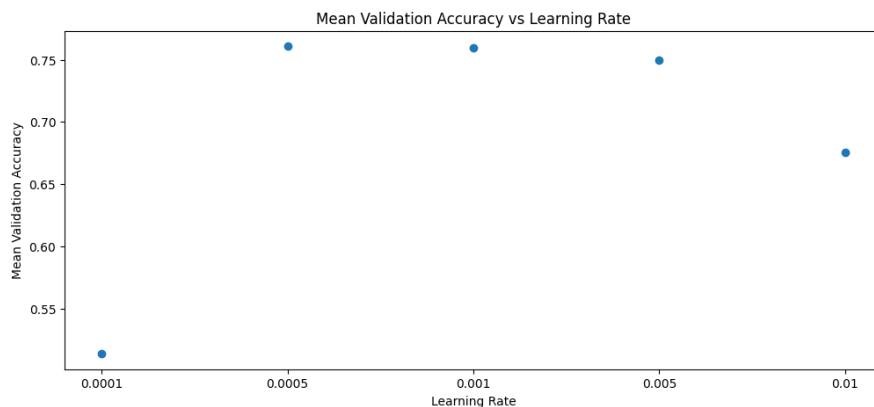


Figure 2: Mean validation plot to determine optimal learning rate

3.2 Result

Final training and testing were conducted using the provided train-validation and test datasets, respectively. Quantitative results showed good classification accuracy, achieving approximately 94% accuracy on the train and 87% accuracy on the test. Moreover, the figure below illustrates plots of the train and test loss and accuracy were analysed to ensure that the trained model had not overfit, with stable convergence and no signs of divergence.

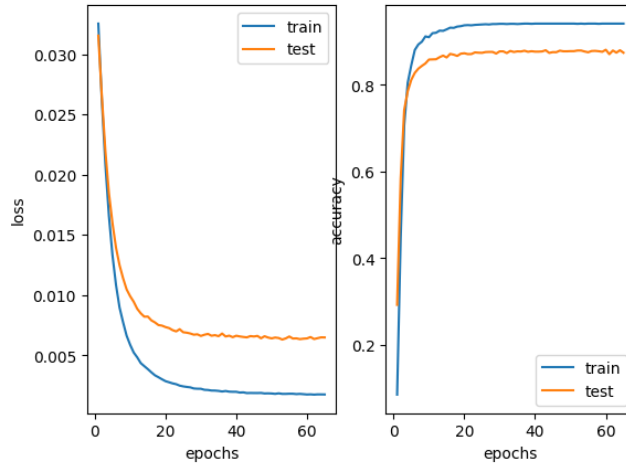


Figure 3: Train and test loss and accuracy plots

This final model will serve as a baseline and strong reference point for evaluating further improvements in the next section.

4 Modifications of Previously Published Architectures

In this section, several architectural modifications to previously published architectures are explored.

4.1 Multi-layer Perceptron

The baseline ResNet-50 model employs a simple linear classifier, which limits the model's ability to learn complex feature interactions as it only learns linear decision boundaries. By adding hidden layers, the MLP can make non-linear decision boundaries, allowing the model to learn complex feature interactions.

A more complex multi-layer perceptron classifier, consisting of a single hidden layer with RELU activations and dropout, replaces the previous classifier head. The maximum number of epochs increased from 100 to 120, accounting for the longer time that may be needed to converge in higher complexity models.

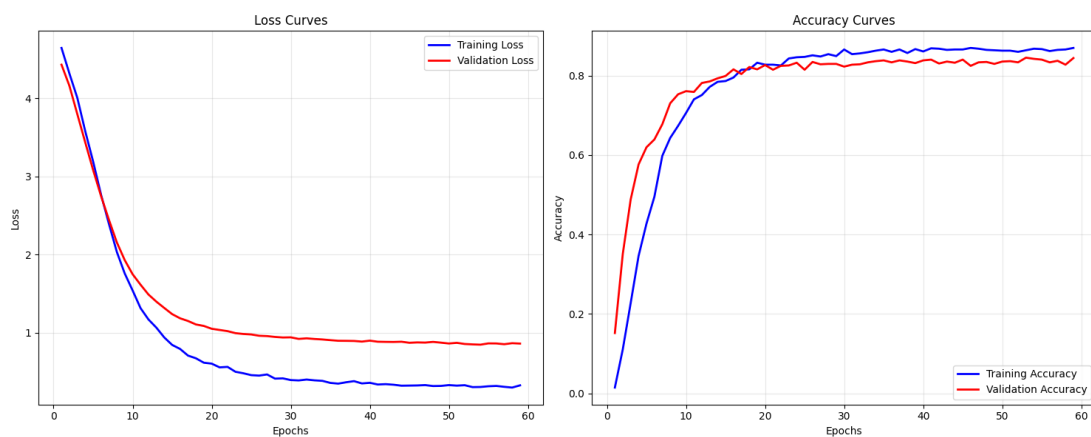


Figure 4: MLP - loss and accuracy plots on train and validation data

4.1.1 Results Analysis

After training, the model achieved a validation accuracy of 84.51% and a test accuracy of 81.44%. The lower value in test accuracy for the model compared to the baseline model demonstrates that

introducing MLP results in worse performance. This is because an additional hidden layer introduces more trainable parameters as compared to the original linear layer. With limited training data, the MLP is highly susceptible to memorising patterns in the training set, leading to overfitting on the training data and poor generalisation on validation and test. As a result, it has lower validation and test accuracy than that of the baseline model.

4.2 Visual Prompt Tuning

Visual Prompt Tuning (VPT) is a parameter-efficient alternative to full fine-tuning for pre-trained vision models. VPT introduces only a small number of trainable parameters in the input space while keeping the entire ResNet-50 backbone frozen. This learned prompt subtly shifts the input distribution in a controlled and task-specific manner, allowing the fixed backbone to produce features that are better aligned with the new task.

4.2.1 Implementation

Visual Prompt Tuning is implemented as follows:

- 1. Start with a pretrained ResNet-50 Model**
 - A pretrained ResNet-50 backbone model is loaded, recognising general patterns, such as edges, textures, and object structures.
- 2. Freeze most of the backbone**
 - Only the last layer and classifier remain trainable.
 - This ensures that strong visual features learned are preserved and prevents the model from overfitting.
- 3. Create a small “learnable” image**
 - The model then creates a small 30x30 RGB patch. This is a learnable tensor with shape 3x30x30 and is initialised with small random values that are close to zero.
 - This prompt is then registered as a model parameter, which means that this prompt will receive gradients and be updated at every training step. The model will also learn the prompt pattern through backpropagation.
 - During training, every pixel in this 3x30x30 is adjusted so that the prompt becomes a task-specific pattern that can help with classification.
- 4. Resize the prompt to match the input image**
 - Since the prompt is only 30x30 but the input images are of 224x224 pixels, it is not possible to add the prompts directly. Therefore, an interpolation algorithm is used to upsample the prompt to 224x224.
- 5. Add prompt to the image**
 - The resized prompt is added to each input image: `prompted_image = original_image + resized_prompt`.
 - Since the prompt values are small, the addition of the prompt acts like a subtle filter. While it does not replace the original image, it slightly shifts the pixel intensities and nudges the pretrained backbone into focusing on task-relevant features.
- 6. Feed the prompted image into ResNet-50**
 - The prompted image is now fed into the ResNet-50 model.
 - Since most layers are frozen, the early convolution layers will extract features exactly as they did while the backbone remains stable and does not overfit.
 - The prompt will reorganise the image so that the frozen features become useful for the task.
 - The unfrozen layer 4 slightly adapts to the task, refining the high-level representations based on the prompted input.
- 7. Only the prompt and the last layer get trained**
 - During backpropagation, the optimiser updates:

- **Visual prompt tensor:** it learns a pattern that steers away from the frozen backbone.
- **Last residual block, layer4:** allows the top of the feature extractor to be more flexible and adapt better.
- **Final fully connected classifier:** must be trained from scratch since the number of output classes has changed.

4.2.2 Results Analysis

The model achieved the highest validation accuracy of 94.02% and test accuracy of 91.04%.

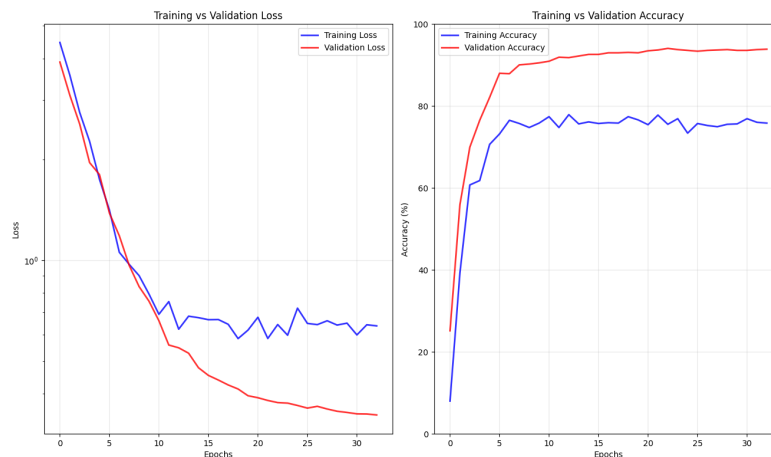


Figure 5: VPT - loss and accuracy plots

The test accuracy of VPT (91.04%) is higher than that of the baseline model (87%), indicating better classification performance. By training only on a small prompt, VPT allows the model to adapt to the input representation while still leveraging the rich pretrained features from the frozen backbone. This provides more flexibility as compared to a fixed classifier and avoids overfitting, leading to a higher accuracy than the baseline model.

4.3 Few-shot Learning

Few-shot learning is a machine learning framework in which the model learns how to make accurate predictions by using limited labelled training data (IBM, n.d.). In many practical applications, data labelling is extremely time-consuming and can take up to 80% of the entire project time (Sapien, 2024). In addition, certain categories like rare diseases or new product types, may also have a limited number of labelled examples available. In such cases, evaluating few-shot learning allows us to investigate if the model can still generalise well with few samples.

A common approach is Metric-based few-shot learning. It classifies new images by comparing them to a small number of training images instead of training on a large classifier. A pretrained model like ResNet-50 is used to extract feature embeddings from images, and in this set up its final layers are fine-tuned on the small support set. For each class, only k support samples are embedded, and their features are then combined to form a class prototype, which acts as a representative feature vector. When a new query image is presented, it is embedded using the pre-trained convolutional layers, and the model will determine its class by measuring how close the query embedding is to each class prototype using a distance metric such as cosine similarity or Euclidean distance.

In the following implementation, Euclidean distance is used as the distance metric to measure the closeness between the query image's embedding and each class prototype. It measures the straight-line distance between 2 feature vectors in the embedding space. This value shows how similar or different the query is from the class prototype, where smaller distances mean greater similarity.

4.3.1 5-shot Learning

In metric-based 5-shot learning, the goal is to classify new images using only 5 labelled examples per class.

4.3.2 Result Analysis

After training, the model achieved validation accuracy of 84.22% and test accuracy of 80.70%.

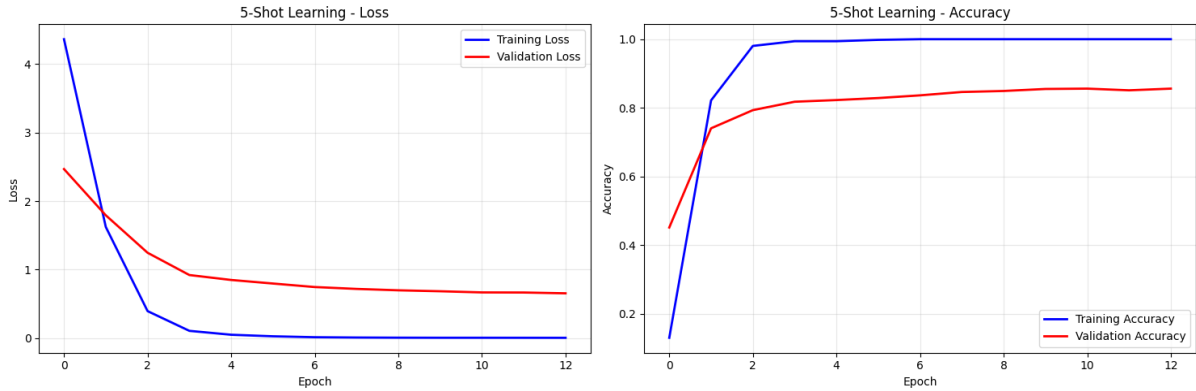


Figure 6: 5-shot learning - loss and accuracy plots

The test accuracy of 80.70% for 5-shot learning is lower than that of the baseline model which achieved 87%. Relying on a small support set results in the model learning both features and noise due to its inability to distinguish between the two in the feature space. As a result, it memorises the 5 support examples rather than learning generalisable features, which leads to overfitting. However, it has to be acknowledged that the model is trained with fewer samples, with the number of epochs required to converge being significantly lesser. Therefore, despite a trade-off of 10% in test accuracy, few-shot learning remains viable due to its practicality and efficiency.

5 Advanced techniques

5.1 Mixup data Augmentation technique

Mixup is a data augmentation technique that can be used to help improve the generalisation ability of machine learning models. It creates new training examples by linearly combining two samples from the dataset. Similarly, the label of the new data sample is the linear combination of the labels of the two chosen data samples. This way, the model receives more data samples for training, improving generalisability. The formulas used to generate a new sample are as follows:

$$\begin{aligned}\hat{x} &= \lambda x_i + (1 - \lambda)x_j, \\ \hat{y} &= \lambda y_i + (1 - \lambda)y_j,\end{aligned}$$

where $\lambda \in [0, 1]$ is a random number

Figure 7: Mixup Formulas for a new sample (Shah, 2020)

5.1.1 Implementation

Since the flowers dataset is balanced across all classes, default selection is used.

The procedure to build a model using Mixup is as follows:

1. Define Mixup function: A function to mix two selected samples is defined before training
2. Call the Mixup function: The function is called in the training loop to generate new samples; the number of new samples generated equals the batch size, which in this case is 32
3. Finding optimal lambda: Experiment with different lambda values [0.2, 0.4, 0.6, 0.8], note that lambda is defined as alpha in the notebook.
4. Final model: Build the Final model using the best alpha found. Training is done on all data from the training dataset and the validation dataset.
5. Final testing: Test on the test dataset while monitoring the accuracy.

5.1.2. Result Analysis

Optima lambda = 0.4 achieved the best validation accuracy of 86.86% and took fewer epochs as compared to other lambdas.

Performance of the final model is as follows:

Best Validation Accuracy	86.86%
Final Test Accuracy	86.52%
Best Test Accuracy	87.32%

Table 1: Performance summary of the final model with Mixup

The lower test accuracy of the model illustrates that it did not perform better than the baseline model. This may be due to a characteristic within the flower's dataset. In the dataset, the differences between classes are not obvious. For example, some species only differ from another by slight details, such as the curvature of the petal, the shape of the stamen, etc. These details are not well-presented by the blended new sample. Instead, these details may be covered up due to the blending, hindering the model from catching these subtle but crucial distinguishing details. Hence, Mixup did not improve the result significantly, potentially due to the similarity of images across the classes in this case.

5.2 Triplet Loss Technique

Triplet Loss is designed for metric learning. It is used to learn a feature space in which similar samples are close together and dissimilar samples are far apart. A triplet consists of the following components:

1. Anchor (A): A reference image of a flower from a specific class
2. Positive (P): Another image from the same class
3. Negative (N): An image from a different class

Euclidean distance is used to calculate the similarity of two samples; the formula is as follows:

$$d(x, y) = \sqrt{\sum_{i=1}^n (y_i - x_i)^2}$$

Figure 8: Euclidean distance formula (TU, 2021)

The triplet loss function that needs to be optimised is as follows:

$$\mathcal{L} = \max(d(a, p) - d(a, n) + \text{margin}, 0)$$

Figure 9: Triplet loss formula (Mohammad, 2018)

where $d(a, p)$ is the Euclidean distance between the anchor and positive image, and $d(a, n)$ is the Euclidean distance between the anchor and negative image.

5.2.1 Implementation

To train the model to learn the embedding layer and classification head, these steps are followed:

1. **Building Base Architecture:**
 - Remove the last classification layer of ResNet50.
 - Add an embedding layer of dimension 512 to output the feature embeddings for each image, which are used for triplet loss calculations.
 - Add a Multi-layer Perceptron classification head.
2. **Triplet Generation:**
 - Generate triplets from training batches using the semi-hard mining strategy.
3. **Embedding Training:**
 - Train the ResNet50 backbone and embedding layer using triplet loss.
 - Keep the classifier frozen during this stage.
 - Monitor embedding quality and stop when triplet loss plateaus
4. **Classification Head Training:**
 - Train the classification head using cross-entropy loss with image labels.
 - Keep backbone and embedding layer frozen during this stage.
5. **Validation and testing:**
 - Evaluate the trained model on the validation dataset and test it on unseen data.

Following is the procedure for semi-hard mining of triplets:

1. **Anchor Selection:** Each image is selected as the anchor once.
2. **Positive Selection:** For each anchor, select one image from the same class as positive.
3. **Negative Selection:** For each anchor, look for semi-hard negatives. These are defined as images that are:
 - Further than the positive but still within the pre-defined margin, i.e., $d(a,p) < d(a,n)$.
 - still within the pre-defined margin, i.e. $d(a,n) < d(A,P) + \alpha$ where α is the margin.
4. **Hardest Negative Selection:** Among the discovered semi-hard negatives, choose the sample that has the closest distance to the anchor (hardest) to represent as the negative of the triplet.

Semi-hard triplets force the model to learn from difficult but informative examples, such as when two images are similar but belong to different classes. Forcing the model to focus on the fine-grained details that distinguish similar images that belong to different classes, hence leading to better feature learning, ultimately leading to more accurate predictions when it encounters new data.

5.2.2 Model Architecture

The final model that is used for validation and testing has the following structure:

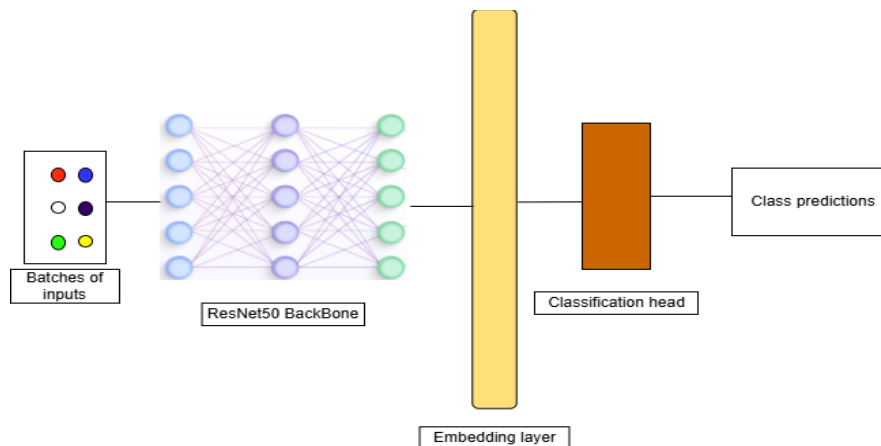


Figure 10: Final Model Architecture with the embedding layer trained with triplet loss

5.2.3 Result Analysis

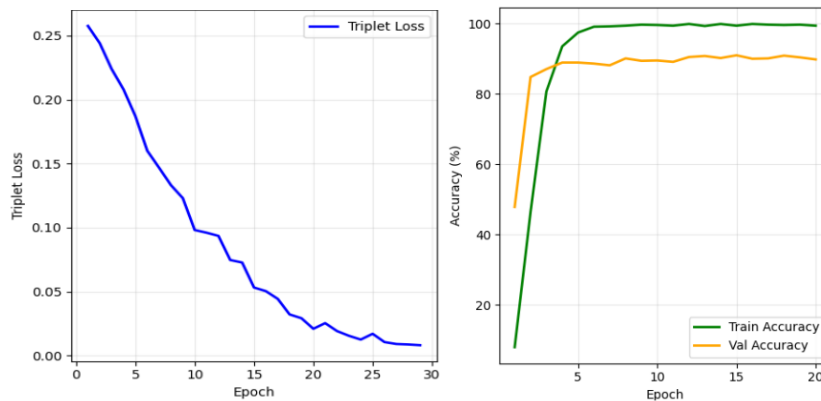


Figure 11: Triplet loss (left) and Training/Validation accuracy (right)

Performance of final model:

Final Train Accuracy	99.41%
Final Validation Accuracy	89.80%
Final test Accuracy	88.08%

Table 2: Performance summary of final model (with triplet loss)

The final test accuracy is 88.08% which is better than the test accuracy of 87% of the baseline model in the earlier part, indicating better classification performance. This is likely due to the embedding layer that is trained with triplet loss, helping the model to learn from generalised and discriminative features rather than from class-specific mappings. This prevents the model from being overfit on class-specific features that appear in the training data. Overall, the newly added embedding layer allows the model to capture small details from the images and is hence able to make more accurate predictions.

6 Discussion

Experiments in Sections 4 and 5 attempt to improve from the baseline transfer learning model through modifications to the existing architecture and learning paradigms. Modifications to the existing architecture include replacing the classifier head with an MLP head and visual prompt tuning. Modifications to existing learning paradigms refer to techniques such as metric-based few-shot learning, Mixup, and Triplet Loss techniques.

For this dataset, the introduction of a hidden layer unnecessarily increases the complexity of the model. However, the small size of the training data and the large number of parameters restrict the model's ability to learn well without overfitting. On the other hand, visual prompt tuning reduces complexity with fewer training parameters, yielding better performance. Therefore, the architectures selected must be chosen with consideration of the size of the dataset, and thus, smaller datasets require simpler models.

Few-shot learning benefits from its practicability and efficiency from using fewer data, albeit with a small trade-off in test accuracy when compared to the baseline model which was trained on twice as many train samples and required significantly higher number of epochs.

Mixup data augmentation did not successfully help the model to pick up these discriminative details, hence leading to the lack of improvement of the model. On the other hand, the embedding layer trained by triplet loss successfully picked up subtle details that help to classify the images more accurately. This highlights the importance of understanding how various techniques may diminish or enhance such subtle and crucial details, and their effectiveness depends on the structure of the data – in this specific example, how similar or dissimilar one class is with respect to another.

6.1 Limitations

6.1.1 Computer resources

The models in this project are built mainly using Google Colab, which provides limited computing resources. For example, for the model using Mixup data augmentation, more lambda values could be tested, such as [0.1,0.2,0.3...,0.9]. However, this would mean building more models and requiring extra computing resources and runtime.

6.1.2 Dataset limitation

Many images feature a significant amount of background noise, such as leaves, soil, and garden fences, which may distract the model from learning flower features. For example, if most of the images from a class show leaves appearing, the model may learn a correlation between the leaves and the class label instead.

Images are shot from various lighting, various angles, and various distances. With 10 images for each class in the training dataset, models may not be able to become robust enough to detect these noises and focus on the flower.

6.2 Future work

After experimenting with Mixup, a more advanced approach is to employ generative models. By training a model such as a GAN (Generative Adversarial Network) or a diffusion model on the existing flower images, new and highly realistic flower images can be generated. These new images can then be added to the original training set, helping to increase its size and diversity. This will help to train a better classification model as the new dataset provides the model with a richer variety of examples to learn from.

In addition, ResNet-50 is used as the backbone for all the models in this project. Another potential aspect to look at is to change the backbone to other pre-trained models, such as VGG or Vision Transformers (ViT), to observe if there is an improvement in results. This could be insightful, especially for few-shot learning since the feature extractions by the backbone played an important role in the training.

7 References

1. GeeksforGeeks. (2024, June 7). *Difference between Traditional Computer Vision Techniques and Deep Learning-based Approaches*. GeeksforGeeks. <https://www.geeksforgeeks.org/computer-vision/difference-between-traditional-computer-vision-techniques-and-deep-learning-based-approaches/>
2. Ginesh. (2019, November 6). *CNN vs MLP for Image Classification*. Medium. <https://medium.com/analytics-vidhya/cnn-convolutional-neural-network-8d0a292b4498>
3. IBM. (n.d). *What is few-shot Learning?* IBM. <https://www.ibm.com/think/topics/few-shot-learning>
4. Mohammad, M. R. (2018, November). *Triplet loss in Pytorch*. Pytorch. <https://discuss.pytorch.org/t/triplet-loss-in-pytorch/30634>
5. Sapien. (2024, January 15). *The Hidden Costs of Data Labeling: Time, Money and Effort*. Sapien. <https://www.sapien.io/blog/the-hidden-costs-of-data-labeling-time-money-and-effort>
6. Shah, K. (2020). *Data Augmentation Tutorial: Basic, Cutout, Mixup*. Kaggle. <https://www.kaggle.com/code/kaushal2896/data-augmentation-tutorial-basic-cutout-mixup>
7. TU. (2021, January 10). *Optimising Pairwise Euclidean distance calculations using Python*. Towards Data Science. <https://towardsdatascience.com/optimising-pairwise-euclidean-distance-calculations-using-python-fc020112c984/>